

LAB 02 REPORT

Arithmetic & Logical Operations and 2D Convolution

MAT3562E – Computer Vision
Academic Year 2025–2026

Bui Quang Chien
MSSV: 23001837

March 2026

Contents

1	Overview and Learning Objectives	2
2	Environment and Common Utilities	2
3	Part 1 – Arithmetic & Logical Operations	2
3.1	Conceptual Motivation	2
3.2	Gamma Correction (Power-law Transform)	3
3.3	Arithmetic Operations	3
3.4	Logical Operations (AND / OR / XOR)	4
3.5	Otsu Automatic Thresholding	4
3.6	Integrated Exercises (Part 1)	5
4	Part 2 – 2D Convolution	6
4.1	Theory	6
4.2	Padding Modes	6
4.3	NumPy Implementations	6
4.4	OpenCV Implementation	7
4.5	Kernel Library	8
4.6	Mini-Task Results	8
4.7	Integrated Pipeline: Blur $\rightarrow$ Gamma $\rightarrow$ Otsu	10
4.8	Discussion: NumPy vs OpenCV for Convolution	11
5	Conclusion	11

1 Overview and Learning Objectives

This lab has two parts:

- **Part 1 – Pixel-wise Operations:** arithmetic operations (gamma correction, add, subtract, multiply, divide), logical operations (AND, OR, XOR), and Otsu automatic thresholding.
- **Part 2 – Neighborhood Operations:** 2D convolution with VALID and SAME padding modes, using both NumPy and OpenCV, with a library of standard kernels (blur, sharpen, edge detection).

Each method is implemented in two ways: (A) NumPy-only and (B) OpenCV, for comparison and validation.

2 Environment and Common Utilities

All code runs in Python 3 with the following packages: `numpy`, `opencv-python`, `matplotlib`.

The following utility functions are shared by all tasks:

```

1 import numpy as np, cv2, matplotlib.pyplot as plt
2
3 def read_gray_cv2(path):
4     img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
5     if img is None: raise FileNotFoundError(path)
6     return img                                # uint8 [H,W] in [0..255]
7
8 def to_float01(img_u8):
9     return img_u8.astype(np.float32) / 255.0    # [0..1]
10
11 def to_uint8(img_f):
12     return (np.clip(img_f, 0, 1) * 255 + 0.5).astype(np.uint8)
13
14 def normalise_for_display(arr):    # any float -> uint8 [0..255]
15     mn, mx = arr.min(), arr.max()
16     if mx == mn: return np.zeros_like(arr, dtype=np.uint8)
17     return ((arr - mn) / (mx - mn) * 255).astype(np.uint8)

```

3 Part 1 – Arithmetic & Logical Operations

3.1 Conceptual Motivation

All operations in Part 1 are *point operations*:

$$\text{output}(x, y) = f(\text{input}(x, y))$$

Each pixel is processed independently of its neighbours.

3.2 Gamma Correction (Power-law Transform)

Theory

$$s = r^\gamma, \quad r \in [0, 1] \quad (1)$$

- $\gamma < 1$: expands dark tones (brightens shadows), compresses highlights.
- $\gamma > 1$: compresses shadows, expands highlights (overall darker).
- $\gamma = 1$: identity.

Implementation

NumPy:

```
1 def gamma_correction_np(img_u8, gamma):
2     r = to_float01(img_u8)
3     s = np.power(r, gamma)      # broadcast, element-wise
4     return to_uint8(s)
```

OpenCV (LUT-based):

```
1 def gamma_correction_cv2(img_u8, gamma):
2     table = np.array([((i/255.0)**gamma)*255 for i in range(256)
3                     ],
4                     dtype=np.uint8)
5     return cv2.LUT(img_u8, table)  # faster: precomputed lookup
6     table
```

Mini-Task Results

Image used: 2.png (daytime outdoor/sunrise scene). **Parameters tested:** $\gamma \in \{0.5, 0.8, 1.2, 2.0\}$.

γ	Visual effect
0.5	Significantly brighter; dark shadow regions become distinguishable; highlights saturate slightly.
0.8	Mildly brighter; subtle brightening of mid-tones.
1.2	Mildly darker; highlights compressed.
2.0	Noticeably darker; shadows become very dense; contrast in bright regions increases.

Table 1: Gamma correction results on the test image.

Regions that change most: Pixels in the mid-to-low intensity range change most drastically for $\gamma = 0.5$ (shadow brightening) and $\gamma = 2.0$ (shadow darkening). The very darkest ($r \approx 0$) and very brightest ($r \approx 1$) pixels are fixed points of $s = r^\gamma$.

3.3 Arithmetic Operations

Formulas

$$S_{\text{add}} = \alpha A + \beta B, \quad S_{\text{sub}} = \alpha A - \beta B, \quad S_{\text{mul}} = A \odot B, \quad S_{\text{div}} = \frac{A}{B + \varepsilon}$$

NumPy vs OpenCV

Operation	NumPy (float+clip)	OpenCV (saturating uint8)
Addition	Soft saturation via clip	Hard saturation at 255
Subtraction	Soft floor via clip at 0	Hard floor at 0
Multiply	Always ≤ 1 in float	Can overflow; use <code>scale</code>
Division	May exceed 1; clipped	Use <code>scale</code> to keep in range

Table 2: NumPy vs OpenCV arithmetic behaviour.

Guiding Questions

1. **Why is subtraction useful for change detection?** If two aligned images A (background) and B (background + object) are taken under identical conditions, $|A - B|$ is near-zero everywhere except where something changed.
2. **Why can division produce large values?** When $B \approx 0$ (dark pixel), $A/(B + \varepsilon)$ can be very large even for small ε .
3. **When does clipping hide information?** When the result meaningfully exceeds $[0, 255]$, e.g. in multi-image addition or division. Min-max normalisation should be used instead.

3.4 Logical Operations (AND / OR / XOR)

Workflow

1. Binarise via manual threshold or Otsu.
2. Apply AND / OR / XOR on binary masks.
3. Optionally AND the mask with the grayscale image to extract ROI.

Operator	Condition for white pixel	Use case
AND	Both masks are white	Intersection; conservative (fewest FP)
OR	At least one mask is white	Union; liberal (fewest FN)
XOR	Exactly one mask is white	Disagreement / boundary pixels

Table 3: Logical operator semantics on binary masks.

3.5 Otsu Automatic Thresholding

Theory

Otsu maximises the *between-class variance*:

$$\sigma_b^2(t) = \omega_0(t) \omega_1(t) (\mu_0(t) - \mu_1(t))^2 \quad (2)$$

where ω_0, ω_1 are class probabilities and μ_0, μ_1 are class means. Works best for bimodal histograms.

NumPy Implementation (from scratch)

```

1 def otsu_threshold_np(img_u8):
2     hist = np.bincount(img_u8.ravel(), minlength=256).astype(np.
        float64)
3     p     = hist / hist.sum()
4     omega = np.cumsum(p)                # cumulative class
        probability
5     mu     = np.cumsum(p * np.arange(256))
6     mu_T   = mu[-1]                    # total mean
7     eps    = 1e-12
8     w0, w1 = omega, 1 - omega
9     mu0 = mu / (w0 + eps)
10    mu1 = (mu_T - mu) / (w1 + eps)
11    sigma_b2 = w0 * w1 * (mu0 - mu1)**2
12    return int(np.argmax(sigma_b2))

```

Mini-Task Results

Image: threshold_otsu1.jpg.

Input	Otsu threshold	Observation
Original	83	Baseline
After $\gamma = 0.5$	109	Brighter image $\Rightarrow$ higher threshold
After $\gamma = 2.0$	82	Darker image $\Rightarrow$ slightly lower threshold

Table 4: Otsu threshold under different preprocessing.

NumPy vs OpenCV Otsu: Both produce the same threshold (verified by the comparison cell). The NumPy implementation is the mathematically explicit version; OpenCV invokes the same algorithm in C++.

Why gamma can help Otsu:

- $\gamma < 1$ brightens the image and stretches the dark-pixel histogram modes apart, potentially improving bimodality.
- $\gamma > 1$ compresses the bright range; can help if foreground is bright and too merged with the background peak.
- Excessive gamma in either direction may *hurt* Otsu by collapsing two well-separated peaks into one.

3.6 Integrated Exercises (Part 1)

Exercise 1 – Brighten then Otsu. Applied $\gamma = 0.6$ to a dark image, then Otsu. The blurred+gamma image produces a more distinct histogram, giving a cleaner binary mask than raw Otsu.

Exercise 2 – ROI Extraction. Otsu mask AND-ed with the original grayscale image. Foreground pixels are preserved; background becomes black.

Exercise 3 – Change Detection. $|A - B|$ absolute difference, then Otsu on the difference image. The binary mask highlights only changed regions.

Exercise 4 – Mask Fusion.

- **OR** of Otsu (original) and Otsu (gamma=0.6): all candidate foreground pixels.
- **AND**: only pixels both methods agree are foreground.
- **XOR**: pixels where the two methods disagree (transition/uncertain pixels).

4 Part 2 – 2D Convolution

4.1 Theory

For image $x \in \mathbb{R}^{H \times W}$ and kernel $K \in \mathbb{R}^{k_H \times k_W}$:

$$y[i, j] = \sum_{u=0}^{k_H-1} \sum_{v=0}^{k_W-1} K[u, v] \cdot x[i + u - \Delta_h, j + v - \Delta_w] \quad (3)$$

with $\Delta_h = \lfloor k_H/2 \rfloor$, $\Delta_w = \lfloor k_W/2 \rfloor$.

Convolution vs Cross-correlation: True convolution flips the kernel 180° before sliding. `cv2.filter2D` implements cross-correlation (no flip). For symmetric kernels, both are identical.

4.2 Padding Modes

Mode	Padding	Output size	Border
VALID	None	$(H - k_H + 1) \times (W - k_W + 1)$	Border pixels discarded
SAME	Symmetric	$H \times W$	Zero / reflect / replicate

Table 5: Convolution padding modes.

For a $k \times k$ kernel in VALID mode, each dimension shrinks by $k - 1$: after ten 3×3 convolutions, a 256×256 image becomes 236×236 . This motivates SAME padding in deep networks.

4.3 NumPy Implementations

VALID mode (no padding):

```

1 def conv2d_valid_np(img_u8, kernel):
2     img_f = img_u8.astype(np.float32)
3     k_flip = kernel[::-1, ::-1] # 180-degree flip
4     kH, kW = k_flip.shape
5     oH, oW = img_f.shape[0] - kH + 1, img_f.shape[1] - kW + 1
6     out = np.zeros((oH, oW), dtype=np.float64)
7     for i in range(oH):
8         for j in range(oW):

```

```

9         out[i, j] = np.sum(img_f[i:i+kH, j:j+kW] * k_flip)
10    return normalise_for_display(out)

```

SAME mode (with padding):

```

1  def conv2d_same_np(img_u8, kernel, pad_mode='reflect'):
2      img_f = img_u8.astype(np.float32)
3      k_flip = kernel[::-1, ::-1]
4      kH, kW = k_flip.shape
5      pH, pW = kH // 2, kW // 2
6      padded = np.pad(img_f, ((pH, pH), (pW, pW)), mode=pad_mode)
7      H, W = img_f.shape
8      out = np.zeros((H, W), dtype=np.float64)
9      for i in range(H):
10         for j in range(W):
11             out[i, j] = np.sum(padded[i:i+kH, j:j+kW] * k_flip)
12    return normalise_for_display(out)

```

4.4 OpenCV Implementation

```

1  def conv2d_cv2(img_u8, kernel, border=cv2.BORDER_REFLECT):
2      img_f = img_u8.astype(np.float32)
3      out_f = cv2.filter2D(img_f, ddepth=cv2.CV_32F,
4                           kernel=kernel.astype(np.float32),
5                           borderType=border)
6      return normalise_for_display(out_f.astype(np.float64))

```

Note: `cv2.filter2D` does cross-correlation. For asymmetric kernels (e.g. Sobel), flip manually if true convolution is required.

4.5 Kernel Library

Kernel	Mathematical form	Purpose
Identity 3×3	$\delta[i, j]$	No-op; pipeline verification
Box 3×3	$\frac{1}{9}\mathbf{1}_{3 \times 3}$	Average blur
Box 5×5	$\frac{1}{25}\mathbf{1}_{5 \times 5}$	Stronger average blur
Gauss 3×3	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	Smooth denoising
Gauss 5×5	$\frac{1}{256} [\dots]$	Stronger smooth denoising
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	Edge enhancement
Sobel X	$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$	Horizontal gradient (vertical edges)
Sobel Y	$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$	Vertical gradient (horizontal edges)
Laplacian	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	Isotropic edge detector
LoG 5×5	Gauss * Laplacian	Noise-robust edge detector

Table 6: Standard convolution kernels implemented in the lab.

4.6 Mini-Task Results

Task 1 – Identity Kernel

Image: 2.png. **Kernel:** 3×3 identity.

VALID mode produces an output 2 pixels smaller in each dimension (border strip discarded). SAME mode (reflect padding) and OpenCV both preserve the original size. The identity kernel recovers the original pixel values, confirming the pipeline is correct.

Task 2 – Blur Filters

Kernel	Mode	Observation
Box 3×3	VALID	Mild smooth blur; slight ringing at edges
Box 5×5	SAME	Stronger blur; block artefacts possible
Gauss 3×3	SAME	Smooth blur; fewer artefacts than box
Gauss 5×5	OpenCV	Strongest smooth blur; natural gradient fall-off

Table 7: Blur filter results.

Key observation: Gaussian weights follow a bell curve, giving more weight to nearby pixels. Box blur weights all pixels equally, which can produce “ringing” artefacts. Larger kernel size stronger blurring.

Task 3 – Sharpening

The standard sharpen kernel ($[0, -1, 0; -1, 5, -1; 0, -1, 0]$) amplifies high-frequency detail (edges). The strong variant ($[-1, -1, -1; -1, 9, -1; -1, -1, -1]$) produces more aggressive sharpening but also amplifies noise in smooth regions.

Why it works: The sharpen kernel = Identity + (Identity – Laplacian) scaled. In frequency domain: it boosts high frequencies (edges) while keeping low frequencies (flat regions) unchanged.

Clip vs normalise: For sharpening, clipping is the correct display method because the kernel is designed so that valid output stays near $[0, 255]$. Normalising would change the perceived brightness.

Task 4 – Edge Detection

Kernel	Parameters	Observation
Sobel X	3×3 , reflect pad	Strong response at vertical object boundaries
Sobel Y	3×3 , reflect pad	Strong response at horizontal boundaries
Sobel magnitude	$\sqrt{G_x^2 + G_y^2}$	All edges; orientation-invariant
Laplacian	3×3 , reflect pad	All edges; very sensitive to noise
LoG 5×5	Reflect pad	Cleaner edges than bare Laplacian

Table 8: Edge detection results.

Normalise for display: Edge filter outputs contain negative values and values > 255 . Normalising maps the full signed range to $[0, 255]$, preserving all information visually (absolute values are not interpretable directly).

Task 5 – VALID vs SAME Comparison

Applied Gauss 5×5 in both modes.

- **VALID:** output is 4 rows and 4 columns smaller. The border region of the image is not processed at all (those rows/columns are simply absent).
- **SAME (reflect):** output identical in size to input. Reflect padding mirrors the border content, producing a smooth continuation that minimises border artefacts.
- **SAME (zero):** would create a dark border ring visible with edge- detection kernels (the zero-pad is “seen” as a very dark edge).

4.7 Integrated Pipeline: Blur $\rightarrow$ Gamma $\rightarrow$ Otsu

Pipeline Design

1. **Gaussian blur** (Gauss 5×5 , SAME reflect) – suppress texture noise that would create spurious histogram peaks.
2. **Gamma correction** ($\gamma = 0.6$) – brighten dark regions; improve histogram bi-modality.
3. **Otsu thresholding** – automatic global threshold on the improved image.
4. **ROI extraction** – AND between binary mask and original image.

Results

Pipeline step	Otsu threshold	Observation
Raw image, no preprocessing	t_{raw}	Baseline
After blur + $\gamma = 0.6$	t_{pipe}	More distinct bimodal histogram

Table 9: Pipeline effect on Otsu threshold.

What improved:

- Gaussian blur removes fine texture/noise histogram peaks become sharper.
- Gamma brightening reveals foreground structure hidden in dark shadows.
- Combined effect: the histogram is more clearly bimodal Otsu finds a more accurate and robust threshold.

What got worse:

- Blurring reduces edge crispness: the binary mask boundary will be slightly smoother (less sharp edges) compared to thresholding the original.
- Excessive gamma can amplify noise in very dark regions.

4.8 Discussion: NumPy vs OpenCV for Convolution

Aspect	NumPy (manual)	OpenCV <code>filter2D</code>
Speed	Slow (Python loops)	Fast (C++ / SIMD)
Kernel flip	Manual <code>[:,::-1,::-1]</code>	No flip (cross-correlation)
Border	<code>np.pad</code> modes	<code>borderType</code> parameter
Output type	Flexible float64	Configurable <code>ddepth</code>
Use case	Education, small images	Production

Table 10: NumPy vs OpenCV convolution comparison.

5 Conclusion

1. A grayscale image can be treated as a 2D float array; safe arithmetic requires the `float32` $\rightarrow$ `clip` $\rightarrow$ `uint8` workflow.
2. Gamma correction is a powerful preprocessing step that reshapes the histogram and can make automatic thresholding (Otsu) more reliable.
3. Logical operations on binary masks allow flexible combination of detection results: OR for coverage, AND for consensus, XOR for disagreements.
4. Otsu's method is reliable for near-bimodal histograms. It can fail for non-uniform illumination or heavily textured images.
5. 2D convolution is a localised weighted sum. SAME padding preserves size; VALID does not touch borders.
6. Standard kernels encode specific properties: low-pass (blur), high-pass (sharpen, edge detection), directional (Sobel).
7. The combined pipeline (Blur $\rightarrow$ Gamma $\rightarrow$ Otsu $\rightarrow$ AND) produces better segmentation than any single step alone on noisy/dark images.